

AAuth / SOGA — Read Ahead for July 10

Sent ahead of our 11:30 conversation. Not a proposal — this is where my thinking is today, so we can spend the time on the open questions rather than background.

Purpose

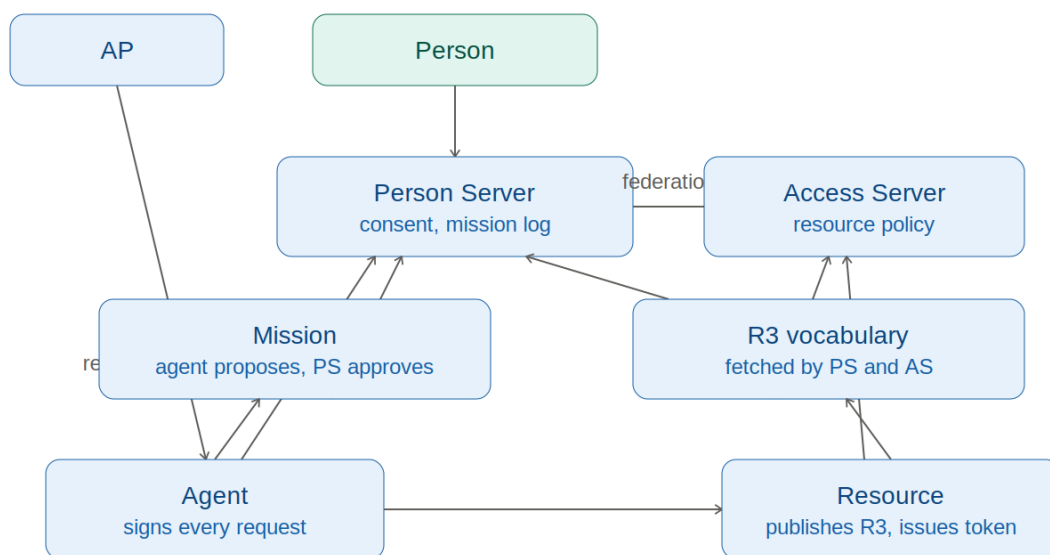
The purpose of this meeting is to better understand the AAuth governance model.

While building a protocol-neutral, execution-time governance reference implementation, I encountered several architectural questions around how delegated authority should be evaluated as circumstances change that appear closely related to concepts in AAuth. Rather than making assumptions, I'd like to validate my understanding of the specification, understand how you envision the Governance Agent and Person Server working together, and identify where our thinking aligns — or differs.

Your architecture, as I understand it from the current draft

AAuth — Full Specification Map (verified)

Deb B Labs — SOGA / AAuth Alignment — 2026-07-09 — Source: draft-hardt-oauth-aauth-protocol-09

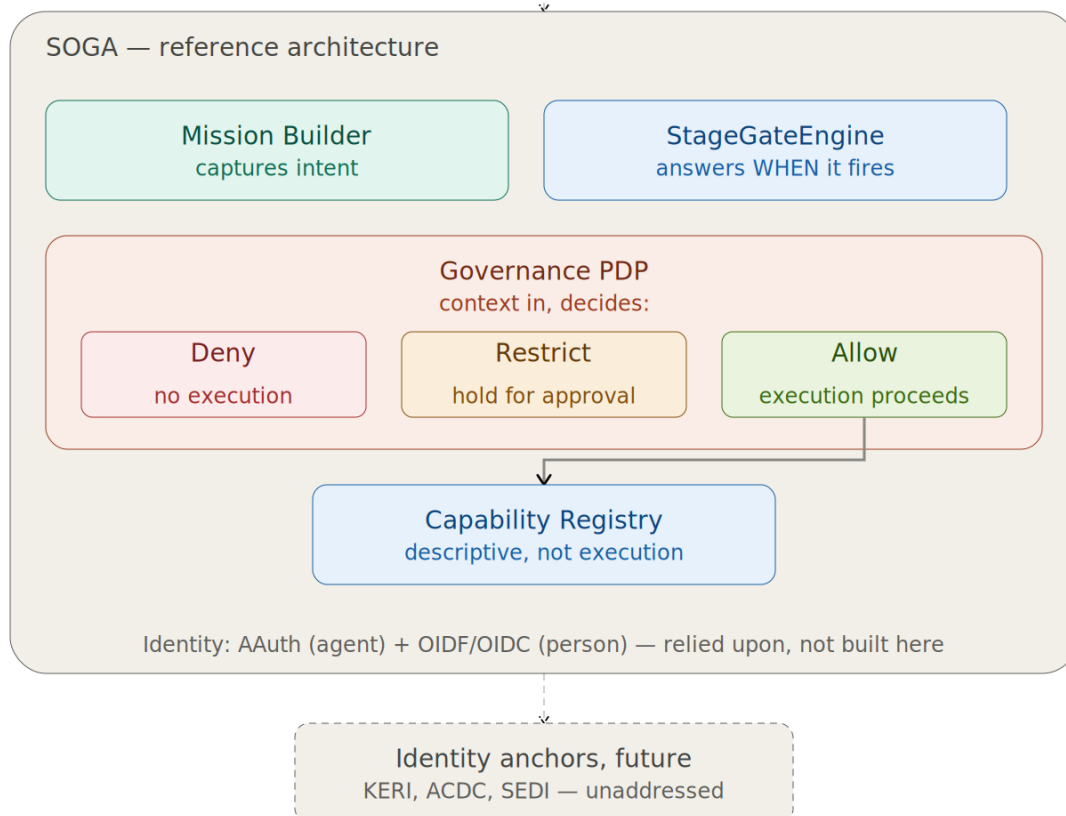
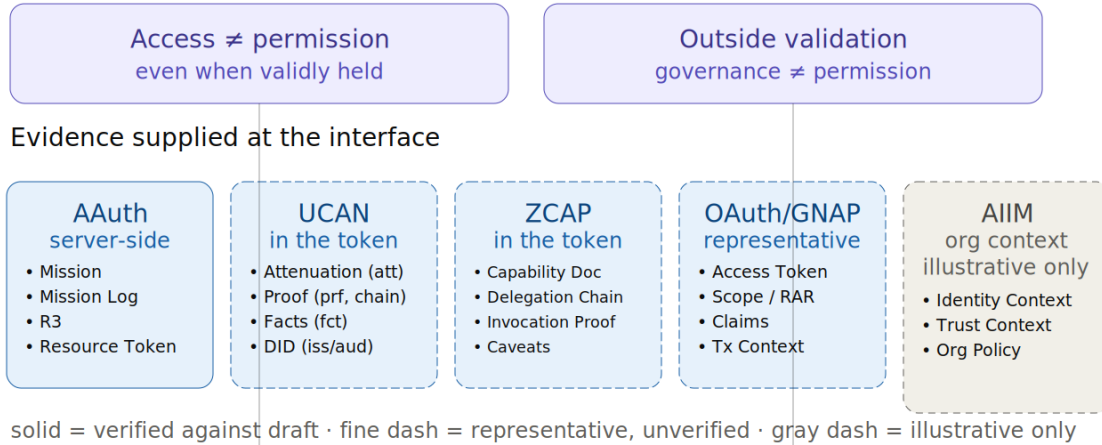


Verified against spec text: Protocol Overview, Person Server, Mission, R3 vocabulary sections. Solid lines = specified relationships.

Five roles and their real relationships, verified directly against the spec — Agent Provider, Person, Person Server, Access Server, Resource.

Ours, for comparison

Why this exists

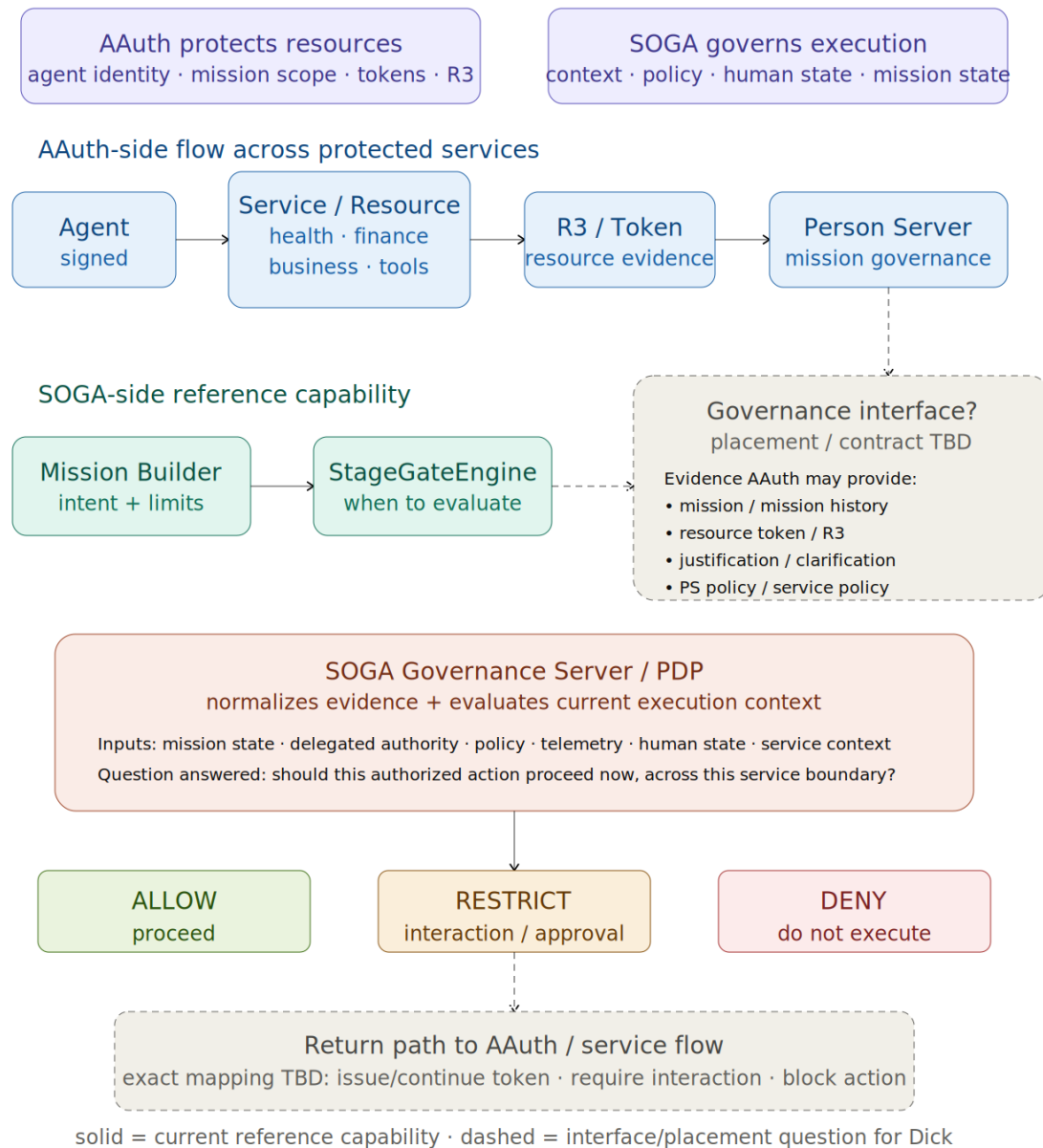


Mission Builder → StageGateEngine → Governance PDP → one of three outcomes (Deny, Restrict, Allow) → Capability Registry. Identity is deliberately out of scope — we rely on AAuth for agent identity and OIDF/OIDC for person identity rather than building our own.

Working Integration Hypothesis

AAuth ↔ SOGA Interface Map

Across services: AAuth authorizes access; SOGA evaluates whether authority should be exercised



This shows where AAuth and SOGA touch: your five roles on top, our architecture below, connected at the points we believe they interact. Our demos run against documented protocol fixtures and produce real governance decisions. What's not built yet is a live identity layer and production deployment — the dashed box marks where that piece connects, and it's what we want to place correctly.

Where our thinking is, and why

We think governance can sit alongside the Person Server, responding on behalf of the person when a stage gate needs approval — evaluating the same inputs you described for your governance agent: resource token and R3, justification, PS policy, mission and history. We add one thing your description

doesn't mention: Subject Agency State, where the same authority produces a different outcome if the person's situation has changed since approval. We'll refer to our own component as the GS, the same way you use PS and AS.

Some missions are designed with authorization checkpoints tagged in advance — a step marked consequential before execution ever reaches it. When execution hits that step, AAuth defers with requirement=interaction, which is what triggers the QR code today. We think that URL should point to our GS instead of the PS. Once GS is consulted, it may determine existing delegated authority is sufficient, or that further interaction is required.

UCAN and ZCAP have no equivalent to a Person Server, so building this as its own service is what lets the same check run across all three protocols without three separate implementations.

Questions

On the governance boundary

1. Does the Governance Agent read PS-held context internally, or can the Person Server provide mission, Mission Log, R3, justification, and policy context to an identified external Governance Server?
2. If an external Governance Server is consulted, how is it identified and trusted by the Person Server, and what context is it authorized to receive?
3. Would requirement=interaction naturally be the protocol mechanism used to defer to that governance capability? Your Resource-Initiated Interaction flow already has the PS chaining a resource's own page into the same handoff — so a URL pointing somewhere other than the PS isn't unprecedented.

On the mission

4. Does AAuth expect to own the mission, or carry a reference to one owned elsewhere? Your spec says the approver is “currently always the PS.” I noticed the word “currently” — was that chosen to leave room for this, or just describing where the draft stands today?
5. How does a mission mark a specific step as consequential in advance? Is there a formal mechanism in the mission object itself, or does that live entirely in Mission Log history and PS judgment at the time of the request?
6. Does Mission Log state persist across every PS instance a mission might touch, or is a PS transaction-scoped in practice? If a mission's history matters to the decision, I want to understand how far that history is expected to travel.

Where this lands

Where this lands changes how we build. If governance has to stay internal to the PS, we rethink our architecture. If it can sit externally, we extend what you've already built.

References

Reference implementation: github.com/DebBLabs/soga (public)

DIF Trusted AI Agents WG:

- Problem Space Report — authored by Alan Karp (identity.foundation/delegated-authority-report/)
- Threat Model for Delegated Authority in AI-Mediated Systems — editor: our team (@debbucci); contributors: Juan Caballero, Alan Karp (identity.foundation/delegated-authority-threat-model/)
- Governance Considerations — also authored by our team (identity.foundation/governance-of-delegated-authority-report/)

Looking forward to the conversation.